

ProcessPilot AI

From Problem to Production

A Full-Stack AI Engineering & Problem-Solving Case Study

Problem → Requirements → Architecture → Implementation → Debugging → Iteration → Testing →
Deployment

Author: Aritra Das

Version: 1.0

Date: September 2026

This case study explains the product from the original problem and product definition through technical design, implementation, failure analysis, testing, and deployment. The emphasis is on engineering reasoning—not only on the final feature list.

0. Document Map

1. Project Overview — What ProcessPilot AI Is
2. The Problem — Why This System Needed to Exist
3. Solution Concept — What the Product Is Designed to Do
4. PRD — Product Requirements Before Implementation
5. TRD — Translating Requirements into Engineering
6. Architecture — Designing the System
7. Database Design — Modeling What the System Must Remember
8. Document Ingestion — Turning Files into Searchable Knowledge
9. PII Protection — Security as a Data-Flow Decision
10. RAG — From Semantic Search to Hybrid Retrieval
11. Embeddings — A Constraint-Driven Infrastructure Decision
12. Multi-Agent Orchestration — Routing Different Kinds of Questions
13. Knowledge Graph — Architectural Evolution
14. Meeting Intelligence — From Unstructured Conversation to Tasks
15. Reliability & Debugging — Problems Encountered
16. Security Architecture — Layered Access Control
17. Observability — Making AI Behavior Inspectable
18. Testing — Verifying Failure Modes
19. Deployment — When Local Assumptions Met Infrastructure
20. Engineering Decision Log
21. Current State & Known Limitations
22. What I Would Do Differently
23. Engineering Lessons

Source discipline: the supplied project draft is treated as the basis. Planned, legacy, and not-verified items are kept explicitly labeled rather than presented as completed capabilities.

1. Project Overview — What ProcessPilot AI Is

ProcessPilot AI is an AI-powered enterprise knowledge and operations platform designed to connect information that normally lives in separate places: organizational documents, meeting outputs, tasks, and organizational relationships.

The system is not simply a chat interface placed in front of an LLM. It is an application built around the model: it ingests and prepares knowledge, retrieves relevant context, answers relational questions through structured data, applies access boundaries, turns meeting information into work, records AI behavior, handles provider failures, and persists application state.

That makes ProcessPilot AI a combination of conventional application engineering and AI engineering. SQL data, vector representations, graph relationships, streamed responses, and probabilistic model output all require different handling.

The documented deployed direction is a React/FastAPI/PostgreSQL-oriented system with hybrid retrieval, a SQL-backed knowledge graph, specialized agent routing, real-time client updates, and PII protection, while explicitly documenting what remains planned, legacy, or unverified.

The core engineering principle: solve the problem first, then choose the architecture and technologies that satisfy the resulting constraints.

2. The Problem — Why This System Needed to Exist

I started with a problem, not a stack.

Organizational knowledge is fragmented. Policies and reference material live in documents. Meetings create decisions and action items. HR and organizational systems define reporting relationships. Tasks represent work that needs to happen. Each source is useful by itself, but useful answers often require information from more than one source.

This creates a systems problem. Relational information is naturally represented in SQL. Document meaning benefits from semantic retrieval. Conversational information arrives as probabilistic LLM output. These representations cannot simply be treated as one data type.

The difficult part was therefore not 'make an LLM answer questions.' It was building the surrounding system that can provide the right information, enforce access boundaries, preserve state, survive deployment constraints, expose failures, and turn AI output into deterministic application behavior.

I decomposed the problem into four connected areas:

- **Knowledge:** ingestion, extraction, PII handling, chunking, embeddings, retrieval.
- **Meetings:** extraction of action items and conversion into tasks.
- **Relations:** explicit organizational hierarchy and relationship queries.
- **Security:** identity, role/department boundaries, and protection of sensitive content.

This decomposition was the first architectural clue: one monolithic chatbot would not cleanly solve every class of problem.

3. Solution Concept — What the Product Is Designed to Do

The solution is a set of cooperating application subsystems rather than one AI endpoint.

Capability	Purpose	System responsibility
Document knowledge	Make organizational files searchable	Validation → extraction → redaction → chunking → embedding → vector persistence
Hybrid retrieval	Answer questions using internal knowledge	Dense vector retrieval plus sparse keyword retrieval
Organizational reasoning	Answer hierarchy and relationship questions	SQL-backed knowledge graph and graph-aware agent
Meeting intelligence	Convert conversation into work	LLM extraction → defensive parsing → tasks
Specialized AI routing	Send questions to the right capability	Controlled orchestration through specialized agents
Security	Prevent unauthorized access / reduce exposure	JWT → RBAC → department/attribute checks → PII handling
Real-time UX	Surface generated work quickly	WebSockets and streamed responses
Operational visibility	Inspect AI behavior	Agent logs, LLM usage, AI failure records

The LLM is therefore one component in a larger application. Retrieval, authorization, persistence, parsing, observability, and deployment behavior remain engineering responsibilities outside the model.

4. PRD — Product Requirements Before Implementation

The first product question was: **what exactly am I building?** A chat interface? A task manager? A document vault? Starting implementation without a boundary would encourage feature growth without a coherent user problem.

The PRD was used to define users, workflows, scope, and boundaries before implementation. The documented roles are Admins, Managers, and Employees.

Problem	Product requirement	Engineering implication
Fragmented documents	Centralized ingestion and retrieval	Document pipeline, embeddings, vector search
Lost meeting actions	Transcript action-item extraction	Structured LLM output + deterministic task persistence
Relational questions	Organizational hierarchy	Knowledge graph / relational relationship model
Unauthorized access	Role and department-aware access	JWT, RBAC, ABAC-style checks
Need for immediate task updates	Real-time delivery	WebSockets
Provider/model failures	Resilient AI integration	Unified LLM client with retry/fallback behavior

What the PRD prevented: building features merely because they were technically interesting; choosing infrastructure before understanding the product; and treating documents, meetings, tasks, and organizational relationships as interchangeable data.

5. TRD — Translating Requirements into Engineering

The TRD translated product requirements into engineering constraints. The principle was: **requirements first, technology second**.

Requirement	Constraint	Technical direction	Reason
RBAC / ABAC	Persistent relational identity and organization context	PostgreSQL	Users, departments and permissions are relational
Vector retrieval	Similarity search alongside application data	pgvector	Relational and vector data can coexist in the database
Real-time tasks	Server push	WebSockets	Avoid repeated HTTP polling
AI flexibility	Providers can fail/change	Unified LLM client	Creates one integration boundary for fallback/retry behavior
Organizational relationships	Hierarchy is not the same as semantic text	Knowledge-graph representation	Makes explicit relationships queryable

This is why the TRD matters: it makes the technology choices explainable. Each major component exists because a product requirement created a constraint that the engineering design had to satisfy.

6. Architecture — Designing the System

The architecture evolved because the first design was useful for prototyping but did not match the deployment environment.

The early architecture described in the supplied draft used React, FastAPI, local Hugging Face embeddings, ChromaDB for vectors, and NetworkX with JSON persistence for graph data.

That design was convenient for local experimentation. Deployment introduced different constraints: the filesystem could not be treated as durable state, and local embedding inference created memory pressure.

Stage	Architecture direction	What it optimized for	What forced change
Prototype	React + FastAPI + local embeddings + ChromaDB + NetworkX/JSON	Fast local experimentation	Persistence and memory constraints
Evolved	React + FastAPI + PostgreSQL/pgvector + SQL-backed graph	Durable application state	Managed persistence and stateless backend behavior
Embeddings	Hosted production path + deterministic development mock	Lower backend memory and development cost	Local model hosting was too costly in memory

Architecture principle: the backend should not depend on local disk as the source of truth for durable application state. Deployment reality turned persistence from an implementation detail into an architectural requirement.

System flow: Client → API → authentication/access checks → specialized application/AI logic → relational/vector/graph data → streamed or real-time response.

7. Database Design — Modeling What the System Must Remember

The database design began with the question: **what information does the system need to remember?**

Data concept	Why it exists
Users	Identity, role, and organizational context
Departments	Organizational boundaries used in access decisions
Documents	Uploaded organizational knowledge
Document chunks	Retrievable pieces of documents
Embeddings	Vector representation for semantic retrieval
Meetings	Persistent meeting information
Tasks	Actionable work extracted from meetings/workflows
Agent logs	AI routing/orchestration observability
LLM usage	Model/token usage tracking
AI failures	Provider/API failure records
KG nodes / edges	Explicit organizational relationships

The supplied draft describes an 18-table SQLAlchemy schema. Because exact model counts have varied across project audits, this final document emphasizes the entity and relationship reasoning instead of using the count as a scale claim.

Department relationships also provide the context required for access decisions. A foreign key by itself should not be described as the complete authorization mechanism; the actual enforcement belongs to the application/data-access implementation.

8. Document Ingestion — Turning Files into Searchable Knowledge

The ingestion problem is: **how do arbitrary organizational documents become searchable AI knowledge?**

File Validation → PyMuPDF Extraction → PII Redaction → Semantic Chunking → Embedding → pgvector Storage

Stage	Engineering purpose
File validation	Reject unsupported or unsafe inputs before expensive processing
Text extraction	Convert PDF content into machine-readable text
PII redaction	Reduce propagation of sensitive values
Semantic chunking	Create retrieval-sized pieces while preserving context
Embedding	Create vector representations for similarity search
Vector persistence	Make generated knowledge available to retrieval

Failure that changed the design: the project draft documents an early asynchronous-processing problem in which the uploaded PDF could disappear before a background thread completed because the deployment filesystem was ephemeral.

Adaptation: extraction was moved into the upload path/in-memory flow so important text could be persisted before the transient file was lost.

Trade-off: large uploads can take longer to process before the request completes. The lesson is that the processing algorithm was not enough; the storage assumption around it also had to match the runtime.

9. PII Protection — Security as a Data-Flow Decision

Sensitive organizational documents create a specific AI boundary problem. If raw HR or similarly sensitive content is passed downstream, the exposure extends beyond the application's own storage.

The documented design therefore handles PII before vectorization. The reasoning is architectural: once sensitive values have propagated into chunks or other downstream representations, removing the original value later does not undo every representation already created.

The implementation described in the draft uses regex-based patterns and considered heavier NLP tooling such as Presidio. The lighter approach was chosen in the context of deployment memory constraints.

Approach	Advantage	Trade-off
Regex redaction	Fast and lightweight	Can miss unusual formats
Heavier NLP detection	Potentially broader entity detection	More runtime/memory complexity
Redaction after retrieval	Late intervention	Does not prevent earlier propagation

Security is therefore part of the data flow, not a UI feature added after the AI pipeline.

10. RAG — From Semantic Search to Hybrid Retrieval

The real RAG question was not 'how do I implement RAG?' It was: **how do I make an LLM answer questions using internal documents reliably?**

The first documented approach used dense semantic retrieval through pgvector cosine similarity. It worked well for naturally phrased questions such as how to request time off.

An identifier-style query exposed a different failure mode. A query such as a specific document ID depends on exact lexical information; semantic similarity is not designed primarily for exact identifier matching.

Iteration	Expectation	Observation	Response
Dense retrieval	Semantic questions should find relevant content	Natural-language policy questions worked	Keep semantic retrieval
Identifier query	Same method should find specific IDs	Exact identifier retrieval was weak	Add sparse keyword retrieval
Hybrid retrieval	Two signals cover different query types	Dense + BM25 are complementary	Continue improving score/rank fusion

The current documented design adds BM25Okapi. Reciprocal Rank Fusion (RRF) is a planned improvement, not an implemented capability. The engineering pattern is: **test** → **observe** → **identify the missing signal** → **make the smallest useful change** → **record the remaining limitation**.

11. Embeddings — A Constraint-Driven Infrastructure Decision

The embedding strategy began with a local-model direction because local inference is convenient during development. The project draft describes an initial plan around a Hugging Face MiniLM-style model.

Deployment changed the calculation. A resident local model added memory pressure to a backend already handling API traffic, retrieval, and application logic.

Decision point	Reasoning
Initial direction	Local embeddings reduce dependency on an external embedding API during development
Constraint discovered	Target deployment memory made resident local inference unattractive
Production direction	Use a hosted embedding path
Development direction	Use a deterministic hash-based mock to reduce repeated API cost during rapid development
Trade-off	Production depends on an external provider; the deterministic mock is a development substitute, not a semantic-quality production embedding

The broader lesson is that model selection is partly an infrastructure decision. The best isolated model is not necessarily the best component for the complete deployed system.

12. Multi-Agent Orchestration — Routing Different Kinds of Questions

Not every enterprise question is a document-search problem. 'Who is the manager of Engineering?' is fundamentally a relationship query. A vector index may retrieve text about Engineering, but the organizational relationship itself belongs in structured data.

The documented architecture therefore uses controlled delegation through a CEOAgent, with specialized capabilities such as SearchAgent, GraphAgent, and SOPAgent.

Question type	Best source	Specialized responsibility
Document / policy question	Document retrieval	SearchAgent / RAG
Organizational relationship	Graph / relational data	GraphAgent
Policy / SOP question	Structured organizational knowledge	SOPAgent

The system deliberately does not claim unrestricted autonomous tool-calling loops. Controlled specialization makes routing behavior easier to constrain, observe, and test. The important engineering decision is separation of responsibility, not the label 'agent'.

13. Knowledge Graph — Architectural Evolution

The knowledge graph began as a prototype-friendly representation using NetworkX with JSON persistence. This made experimentation with nodes and relationships straightforward.

Deployment exposed a different requirement: graph state had to survive a stateless backend and remain safe to update. A local JSON file was not a dependable source of truth in that environment.

The graph was migrated into relational tables for nodes and edges. Organizational hierarchy can now be reconstructed through SQL while sharing durable persistence with the rest of the application.

Aspect	Before	After
Representation	NetworkX in memory	SQL tables: kg_nodes / kg_edges
Persistence	JSON file	Database-backed state
Primary advantage	Fast prototyping	Durable persistence and transactional relational storage
Trade-off	Simple graph-library algorithms	Less native graph-library functionality

This is a direct example of deployment changing architecture: the prototype was not discarded because it was conceptually useless; it was replaced because its persistence model did not match the runtime.

14. Meeting Intelligence — From Unstructured Conversation to Tasks

Meeting transcripts are unstructured, while the application needs predictable structures before it can create database Tasks.

Transcript → LLM extraction → structured payload → Task records → real-time client update

The documented implementation prompts the LLM for structured action items and uses defensive parsing around the response before persisting tasks. The goal is to prevent conversational filler or formatting variation from flowing directly into deterministic database operations.

This creates a useful boundary between probabilistic and deterministic systems: the model interprets the transcript, while application code decides what constitutes a valid structured record and how that record changes persistent state.

The same pattern appears elsewhere in ProcessPilot AI: let the model perform interpretation, but keep authorization, persistence, validation, and side effects under deterministic application control.

15. Reliability & Debugging — Problems Encountered

Two concrete issues documented in the project illustrate different classes of failure.

Bug 1 — LLM model deprecation / 404

Symptom: AI requests began failing and surfaced as server errors.

Investigation: the configured Groq model identifier had become unavailable.

Root cause: the application contained a hardcoded provider/model assumption.

Fix: the LLM client was changed to detect provider-side model availability and retry through an available model path.

Lesson: external AI dependencies should be isolated behind a client boundary.

Bug 2 — Timezone mismatch

Symptom: the Analytics page failed while processing date values.

Investigation: database timestamps were timezone-aware while locally generated timestamps were naive, making Python comparisons/sorting unsafe.

Root cause: inconsistent datetime conventions across system boundaries.

Fix: normalize timestamps to UTC before sorting and analytics operations.

Lesson: time should be treated as a system-wide data contract.

16. Security Architecture — Layered Access Control

Request → JWT authentication → RBAC → department/attribute access checks → PII handling → downstream retrieval/storage

Layer	Question it answers
JWT authentication	Who is making the request?
RBAC	What role-based capabilities does this identity have?
Department / attribute checks	Is this user allowed to access this organizational scope?
PII redaction	What sensitive content should not propagate downstream?
Audit / operational records	Can important behavior be investigated later?

Authentication, authorization, and data protection solve different problems. A valid JWT does not mean the user can access every document, and an authorized user does not imply that sensitive values should be passed unchanged into an external model boundary.

The project material describes department-aware access control and PII handling. Formal compliance claims such as SOC 2 or GDPR certification are explicitly not verified.

17. Observability — Making AI Behavior Inspectable

AI systems introduce a failure mode that ordinary API monitoring can miss: an HTTP 200 response can still contain a poor, incorrectly routed, or weakly grounded answer.

Record	Why it matters
agent_logs	Shows routing/orchestration behavior so incorrect agent selection can be investigated
llm_usage	Tracks token/model usage so cost patterns are visible
ai_failures	Records provider/API failures that may otherwise be hidden behind outer application responses

The observability question therefore becomes not only 'did the endpoint return 200?' but 'what did the AI system actually do, and can I reconstruct why?'

18. Testing — Verifying Failure Modes

Testing was framed around failure modes rather than simply listing endpoints.

Area	Potential failure	Verification goal
ABAC / department access	Cross-department data leakage	Unauthorized department queries should be rejected
Document ingestion	Invalid extraction / processing behavior	Pipeline behavior should remain isolated and predictable
AI integration	Provider/model failure	Failure-handling path should be exercised
Analytics	Timezone mismatch	Normalized datetimes should remain comparable

Testing principle: if a design decision exists because something can fail, that failure should eventually become a testable scenario.

19. Deployment — When Local Assumptions Met Infrastructure

Deployment became architectural feedback rather than a final checklist item.

Local assumption	Deployment reality	Response
Local disk can hold state	Filesystem is ephemeral	Move durable state into managed persistence
Local model can remain resident	Backend memory is constrained	Use hosted production embeddings and lightweight development substitute
File-backed graph state is acceptable	Backend may restart	Move graph state into database tables
Background processing can rely on uploaded file	Transient file may disappear	Persist critical extracted content before it is lost

The central lesson is: **deployment constraints are architecture constraints**. A design that works on a developer machine can still be wrong for its target runtime.

20. Engineering Decision Log

Problem	Decision	Trade-off	Documented state
Graph persistence	Move file-backed prototype to SQL tables	Less native graph-library functionality	SQL-backed kg_nodes / kg_edges
LLM model/provider changes	Centralize provider interaction and fallback/retry	Extra logic and possible first-failure latency	More resilient integration boundary
Exact document identifiers	Add BM25 alongside dense retrieval	More retrieval complexity	Hybrid retrieval; RRF planned
Sensitive document content	Redact before vectorization	Regex may miss edge cases	Lightweight PII redaction
Deployment memory	Avoid resident local embedding model	Hosted dependency	Production provider + deterministic development mock
Ephemeral uploads	Persist critical extracted content before transient loss	Potentially slower upload path	Safer ingestion under documented constraint

The decision log is the shortest representation of the project's evolution: each change began with a concrete problem or constraint, not with a desire to add another technology.

21. Current State & Known Limitations

The project draft describes the current system as a deployed React/FastAPI/PostgreSQL-oriented application with hybrid retrieval, a SQL-backed knowledge graph, specialized multi-agent routing, WebSockets, and PII protection.

Capability	Status	Meaning
Hybrid retrieval	Implemented	Dense semantic retrieval is complemented by BM25 keyword retrieval
RRF	Not implemented	Identified as a future improvement to hybrid rank/score merging
Autonomous tool loops	Not implemented	System uses controlled specialization/routing
Formal SOC 2 / GDPR	Not verified	No formal certification/compliance evidence is claimed
ChromaDB	Legacy	Associated with the early architecture; superseded by the database-oriented direction

A credible engineering document should make these boundaries visible. A planned capability is not the same thing as an implemented one, and a legacy component is not the same thing as the current architecture.

22. What I Would Do Differently

1. Start with durable object storage earlier. The ingestion flow spent engineering effort adapting around ephemeral local files. Separating uploaded-file persistence from compute earlier would make background processing easier to reason about.

2. Introduce retrieval evaluation earlier. Manual query testing was valuable for discovering the dense-retrieval weakness, but an evaluation framework introduced earlier would make retrieval changes easier to compare objectively.

These are lessons from implementation, not claims that the current architecture is invalid.

23. Engineering Lessons

1. Architecture should respond to constraints

The graph migration and embedding decision show that a convenient component can become the wrong component once persistence, memory, and deployment requirements are real.

2. Security belongs in the data flow

PII protection is most useful before sensitive content propagates into downstream representations and model boundaries.

3. AI systems need deterministic application control

LLMs are probabilistic. Database writes, access decisions, validation, and task creation are application responsibilities. Explicit boundaries make the overall system easier to reason about.

4. Retrieval is an engineering problem

Dense retrieval and keyword retrieval solve different failure modes. The useful workflow is to test representative queries, observe where an approach breaks, and add the missing signal.

5. Observability must cover behavior

A successful API response does not prove a useful AI result. Routing, usage, and failure records make the system inspectable.

6. Deployment is part of development

The runtime environment exposed assumptions about storage, memory, and background processing. Those observations directly changed the architecture.

The engineering mindset

I did not start with: **'Which AI technologies should I use?'**

I started with: **'What problem am I trying to solve?'**

Then: What information does the system need? What architecture supports it? What can fail? How do I protect the data? How do I observe failures? How do I test it? How does it behave under deployment constraints? What should change after learning from the implementation?

ProcessPilot AI is the result of that sequence: problem → requirements → design → implementation → failure → investigation → adaptation → deployment.

Appendix. How to Read This Case Study

The document intentionally distinguishes three kinds of statements:

- **Implemented:** presented as part of the current system when supported by the supplied material.
- **Planned / legacy:** explicitly labeled rather than presented as current functionality.
- **Not verified:** not promoted to a completed claim when the supplied material does not establish evidence.

The goal is not to make the project sound larger than it is. The goal is to make the engineering work understandable: what the system was trying to solve, why a design was chosen, what changed when assumptions failed, and what remains to be improved.

ProcessPilot AI — From Problem to Production

Aritra Das · Version 1.0 · September 2026